

Dart Asenkron Programlama- Proje Ödevi

Üniversite: Erciyes Üniversitesi

Fakülte: Mühendislik Fakültesi

Bölüm: Bilgisayar Mühendisliği

Ders: Mobile Application Development

Öğretim Üyesi: Dr. Öğr. Üyesi Fehim Köylü

1. Asenkron Programlama Nedir?

Asenkron programlama, programların aynı anda birden fazla işlemi gerçekleştirmesini sağlar. Bu, özellikle giriş/çıkış (I/O) işlemlerinin beklenmesi gerektiğinde kullanılır. Senkron işlemler, her işlem tamamlandığında bir sonraki adıma geçerken, asenkron işlemler, işlemlerin tamamlanmasını beklemeden devam eder ve sonuçlar geldiğinde işlenir.

Örnek Senkron Kod:

```
void main() {  
    print('İşlem 1 başladı');  
    print('İşlem 1 tamamlandı');  
    print('İşlem 2 başladı');  
    print('İşlem 2 tamamlandı');  
}
```

Örnek Asenkron Kod:

```
void main() async {  
    print('İşlem 1 başladı');  
    await Future.delayed(Duration(seconds: 2));  
    print('İşlem 1 tamamlandı');  
  
    print('İşlem 2 başladı');  
    await Future.delayed(Duration(seconds: 1));  
    print('İşlem 2 tamamlandı');  
}
```

2. Dart'ta Asenkron Programlama Temelleri

Dart, asenkron programlamayı Future ve Stream sınıfları ile sağlar. Asenkron fonksiyonlar, genellikle async ve await anahtar kelimeleriyle tanımlanır.

2.1 Future ve Stream Nedir?

- Future: Bir işlemin sonucunu bekleyen bir yapıdır. Bir işlemin bitip bitmediğini kontrol etmemize yardımcı olur.
- Stream: Zaman içinde gelen veri akışlarını temsil eder. Bu, sürekli veri üretmek veya birden fazla veri sonucu almak için kullanılır.

2.2 Async ve Await Anahtar Kelimeleri

async anahtar kelimesi bir fonksiyonu asenkron hale getirir. await ise bir Future'ın tamamlanmasını bekler.

Basit Bir Future Kullanımı Örneği:

```
void main() async {  
    print('İşlem Başlıyor');  
    await Future.delayed(Duration(seconds: 2));  
    print('İşlem Tamamlandı');  
}
```

3. Future Kullanımı

Future, bir işlemin sonucunu temsil eder ve async fonksiyonları içinde kullanılır.

3.1 Future ile Asenkron İşlem Yapma

```
Future<String> fetchData() async {  
    await Future.delayed(Duration(seconds: 3)); // Simülasyon için bekleme  
    return 'Veri alındı';  
}
```

```
void main() async {  
    print('Veri alınıyor...');  
    String data = await fetchData(); // Future bekleniyor  
    print(data); // 'Veri alındı'  
}
```

3.2 then() ve catchError() Metodları

then() metodu, bir Future tamamlandığında çalıştırılacak bir işlev belirler. catchError() ise hata yönetimi sağlar.

```
void main() {  
    Future.delayed(Duration(seconds: 2), () {
```

```

        throw 'Bir hata oluştu!';
    }).then((_) {
        print('İşlem başarıyla tamamlandı');
    }).catchError((e) {
        print('Hata: $e');
    });
}

```

4. Stream Kullanımı

Stream, birden fazla asenkron veriyi sırayla almanızı sağlar. Stream genellikle veri akışlarının yönetilmesinde kullanılır.

4.1 Stream ve StreamSubscription

Bir Stream, veri akışı başlatıldığında farklı verilere ulaşmanızı sağlar. StreamSubscription, bu akışı dinler.

```

void main() {
    Stream<int> counterStream = Stream.periodic(Duration(seconds: 1), (x) => x);
    var subscription = counterStream.take(5).listen((data) {
        print('Veri: $data');
    });
}

```

4.2 Stream ve await for

await for kullanarak bir Stream'in her verisini sırasıyla alabilirsiniz.

```

Stream<int> countNumbers() async* {
    for (int i = 0; i < 5; i++) {
        await Future.delayed(Duration(seconds: 1));
        yield i;
    }
}

```

```

void main() async {
    await for (var number in countNumbers()) {
        print('Sayac: $number');
    }
}

```

```
}
```

5. Error Handling (Hata Yönetimi)

Asenkron işlemlerde hata yönetimi önemlidir. Dart, try-catch blokları ile asenkron hataları yönetmenize olanak tanır.

```
Future<void> fetchData() async {  
  try {  
    await Future.delayed(Duration(seconds: 2));  
    throw 'Bir hata oluştu';  
  } catch (e) {  
    print('Hata: $e');  
  }  
}
```

```
void main() {  
  fetchData();  
}
```

6. Performans ve Verimlilik

Asenkron programlama, özellikle uzun süren I/O işlemleri sırasında uygulamanın yanıt vermeye devam etmesini sağlar. Ancak, aşırı asenkron işlem kullanımı da performans sorunlarına yol açabilir.

6.1 İyi Uygulamalar

- Gereksiz yere çok fazla Future çalıştırmak yerine işlemleri gruplandırarak başlatmak.
- Stream kullanarak veri akışlarını verimli bir şekilde yönetmek.

7. Sonuç ve Öneriler

Asenkron programlama, uygulamaların performansını artırmak ve yanıt verme sürelerini iyileştirmek için oldukça önemlidir. Dart, Future ve Stream sınıfları ile asenkron programlama konusunda güçlü araçlar sunmaktadır. İyi bir hata yönetimi ve performans optimizasyonu ile asenkron programlama çok daha verimli hale getirilebilir.